ReviewPulse

Turn hundreds of customer reviews into clear actions

Project documentation: overview, user guide, technical report, data and AI prompts log

Advanced AI Algorithms, ESSEC Business School, Advanced Master in Marketing Management & Digital

Team: [names of the team members]

October 2026

Code and data: https://github.com/Monka31/reviewpulse

Run in Colab: https://colab.research.google.com/github/Monka31/reviewpulse/blob/main/ReviewPulse.ipynb

Published reports: https://monka31.github.io/reviewpulse/

Demo video: [link to the video]

Overview

Live reports: https://monka31.github.io/reviewpulse/

ReviewPulse reads every Google review of a business and gives its owner, in a couple of minutes, a dashboard that answers four questions. What do customers like? What do they complain about, and is it getting better or worse? Which complaints were never answered? What should the owner reply to each of them?

The application runs in Google Colab on a free GPU, with three open-source models downloaded from Hugging Face. It needs no paid API and no API key.

The problem

A restaurant, a shop, a hotel or a school can collect several hundred reviews, and new ones arrive every week. Owners rarely have time to read them, so they look at the star average and move on. Recurring problems go unnoticed, and complaints stay unanswered even though every future customer can read them on Google Maps.

The ESSEC Cergy campus, one of our two demo cases, shows how misleading the average can be. Its public rating is 4.28 out of 5 over 250 ratings. When we keep only the 76 people who wrote a text, the average drops to 3.66, and 22 of those 76 written reviews give a single star. The file contains no reply from the school to any of them. The yearly trend also dipped in 2023 (3.9 out of 5, with 28% of ratings at one or two stars) before recovering in 2026.

What the application does

The user drops an Outscraper export (Excel or CSV) of a business's Google reviews, picks the type of business and clicks Analyze reviews. ReviewPulse then detects the sentiment of every written review, assigns each review to one or several themes adapted to the business (food quality and waiting time for a restaurant, admissions and campus for a school), and computes statistics per theme and over time.

A language model writes a short diagnosis with strengths, points to improve and three priority actions, and drafts a public reply for each unanswered complaint, in the language of the review. Customers' own star ratings are then used to measure how often the AI reads a review correctly.

Everything is shown on one dashboard: the main figures, the AI diagnosis, five charts (sentiment, share of themes, rating over time, a priority matrix showing which themes to fix first, and average stars by theme), the list of unanswered complaints with their reply drafts, and a table of every review. Filters by period, stars and theme update the whole dashboard instantly, without running the models again. The same dashboard can be exported as a static web page and published on GitHub Pages, which gives each analysis a permanent link.

Value if widely used

Large chains pay customer-experience platforms and consultants for this kind of analysis. Independent businesses, which make up most of the restaurants, shops and hotels people review, usually do it by hand or not at all. ReviewPulse gives them the same reading of their reviews for free, in minutes, with open models they can run themselves. Used widely, it would save owners the hours spent reading reviews, point them to the problems customers mention most, and help them answer unhappy customers while the answer can still change their mind. Customers would benefit too, through faster fixes and businesses that visibly respond.

Quick start

Open the notebook with the Open in Colab button, choose `Runtime > Change runtime type > T4 GPU`, then `Runtime > Run all`. After three to five minutes the last cell prints a `gradio.live` link: that is the application. Drop a review file in it, or use the ESSEC dataset already in the project.

The user guide explains every screen, the local installation and how to collect reviews for any business. Section 8 of the notebook also generates the narrated demo video automatically, from the same analysis.

How it works

Sentiment comes from `nlptown/bert-base-multilingual-uncased-sentiment`, a BERT model fine-tuned on product reviews in six languages, French and English included, which predicts a rating from one to five stars. Themes come from `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli`, a multilingual natural language inference model used for zero-shot classification, so a business can define its own themes without labeling a single example. Text generation uses `Qwen/Qwen2.5-3B-Instruct` on a GPU and the smaller `Qwen2.5-0.5B-Instruct` on a CPU. The technical report explains each choice, the prompts and the evaluation method.

How the project answers the assignment

The application is built from existing tools, as the assignment asks: pre-trained models from Hugging Face, code and hosting on GitHub (repository, Colab link and GitHub Pages), and the Python ecosystem around them. It goes beyond the course content by chaining three transformer models (classification, zero-shot inference and generation) in one pipeline and by evaluating them against real labels. It addresses a specific need, the unread and unanswered reviews of small businesses, and it can be re-run by anyone in one click from the Colab notebook, on the data stored in this repository. The prompts log reports where and how AI assistants were used, with the prompts. The data sources are described in DATA.md.

The project applies several concepts from the course. The sentiment model is supervised learning and transfer learning: a model trained on millions of reviews is reused without retraining. The theme detector shows zero-shot learning through natural language inference. The diagnosis and replies use generative AI with prompt engineering and grounding, since the model receives computed statistics and real excerpts instead of being asked to read everything. The evaluation uses accuracy against a majority-class baseline, precision, recall and a confusion matrix. Finally, the design keeps a human in the loop (replies are drafts) and removes reviewers' personal data on load.

Repository

```
reviewpulse/
├── ReviewPulse.ipynb          one-click Colab notebook (analysis and app)
├── app.py                     Gradio web application
├── requirements.txt           Python dependencies
├── reviewpulse/               analysis package
│   ├── config.py              models, theme presets and settings
│   ├── data.py                file loading, column detection, anonymization
│   ├── models.py              Hugging Face models (and a mock backend for tests)
│   ├── analysis.py            sentiment, themes, statistics
│   ├── llm.py                 every prompt sent to the language model
│   ├── evaluation.py          accuracy against customer stars
│   ├── dashboard.py           HTML dashboard, charts and filters
│   ├── charts.py              interactive Plotly charts for the notebook
│   ├── report.py              static reports for GitHub Pages
│   └── demo_video.py          automated narrated demo video
├── scripts/
│   ├── analyze.py                     command line: review file to report
│   ├── make_demo_video.py             command line: build the demo video
│   └── collect_reviews_outscraper.py  review collection and anonymization
├── data/                      anonymized review datasets
├── docs/                      GitHub Pages site with the published reports
├── documentation/             user guide, technical report, data, prompts, video script
└── tests/                     automated tests (no model download needed)
```

Limitations

The sentiment model sometimes misreads irony and mixed reviews, especially at three stars. The evaluation section of each analysis shows its measured accuracy on that business. Zero-shot themes depend on how the themes are worded, and vague themes give noisy results. The language model can be generic or occasionally wrong, so every reply is a draft that a person must read before posting. Collecting Google Maps reviews with a third-party service may conflict with Google's terms of service; we used the data for analysis only, kept no reviewer identity, and the project should not be read as a commercial scraping service. Free Colab GPUs are not guaranteed, and on a CPU the generation steps are slow.

Credits

Models: nlptown/bert-base-multilingual-uncased-sentiment, MoritzLaurer/mDeBERTa-v3-base-mnli-xnli, Qwen/Qwen2.5-3B-Instruct and Qwen/Qwen2.5-0.5B-Instruct.

Packages: Transformers, Accelerate, PyTorch, Gradio, pandas, Plotly, Python-Markdown, openpyxl, SentencePiece and pytest. Fonts: Inter and Plus Jakarta Sans from Google Fonts.

Data: Google Maps reviews collected with Outscraper, see DATA.md.

Team: [names of the team members]. Course: Advanced AI Algorithms, ESSEC Business School, Advanced Master in Marketing Management & Digital, 2026. License: MIT.

User guide

This guide explains how to start ReviewPulse, how to analyze the reviews of a business, how to read each part of the dashboard and how to publish a report. You do not need to know how to program to use the application through Google Colab.

1. What you need

You need a Google account to use Colab, and a file of reviews. The project already contains the reviews of the ESSEC Cergy campus (`data/reviews_ESSEC_Cergy.csv`), so you can try the application without collecting anything. To analyze another business, section 6 explains how to export its Google reviews with Outscraper.

2. Start the application in Google Colab

Open the notebook at https://colab.research.google.com/github/Monka31/reviewpulse/blob/main/ReviewPulse.ipynb and sign in with your Google account.

Before running anything, choose a GPU: open `Runtime > Change runtime type`, select T4 GPU and click Save. The application also works without a GPU, but writing the diagnosis and the replies then takes several minutes instead of seconds.

Then click `Runtime > Run all`. If Colab warns that the notebook was not written by Google, click Run anyway. The notebook checks the GPU, downloads the code from GitHub, installs the libraries, downloads the three models (about two minutes the first time), analyzes the ESSEC reviews and displays the dashboard inside the notebook. Its last cell starts the web application and prints a link that looks like `https://xxxx.gradio.live`. Open that link: the application runs in your browser, on any computer or phone.

The link works for about 72 hours, as long as the Colab tab stays open and active. When Colab disconnects, run the last cell again to get a new link. If something looks out of date or broken, use `Runtime > Disconnect and delete runtime`, then `Runtime > Run all`. This starts from a clean copy of the latest code.

A red message mentioning `diffusers` and `huggingface-hub` may appear during installation. It concerns a library preinstalled on Colab that ReviewPulse does not use, and you can ignore it.

3. Run an analysis

The New analysis panel on the left holds every setting.

Drop the review file in the Review file area. It accepts the Excel or CSV file exported by Outscraper as it is, and any other CSV or Excel file with a column of review text. If you do not drop a file, the application uses the dataset selected in the list below it. A dropped file always takes priority over the list.

Type the name of the business as you want it to appear on the dashboard. When this field is empty, the name is read from the file.

Choose the business type: Restaurant, School / University, Hotel, Retail store or Generic business. Each type loads a list of themes adapted to it. For a school, for example, the themes are teaching quality, campus and facilities, admissions, staff and welcome, administration, student life, career opportunities and tuition fees. You can edit the list in the Themes to detect field (comma-separated, ten themes at most). Short and concrete themes work best, such as "waiting time" rather than "experience".

Choose the language of the AI diagnosis, EN or FR. Reply drafts are always written in the language of the review they answer.

Advanced settings holds three options. Max reviews limits the analysis to the most recent written reviews (500 by default). Reply drafts written during the analysis sets how many of the latest unanswered complaints get a reply draft straight away (5 by default; the others can be drafted on demand). Write the AI diagnosis can be unticked for a faster analysis without text generation.

Click Analyze reviews. A progress bar shows each step. On a T4 GPU, 300 reviews should take about one to two minutes.

4. Read the dashboard

The header shows the business type and name, the number of written reviews analyzed, the number of star-only ratings (people who gave stars without writing) and the period covered.

The four main figures come next. Average rating counts every rating in the file, star-only ones included, so it matches what Google shows. Written reviews gives the number of reviews with text and their own average, which is often lower because unhappy customers are more likely to write. Negative reviews is the share of written reviews the AI reads as negative. Complaints left unanswered counts negative reviews with no reply from the owner, together with the reply rate on complaints.

The AI diagnosis is written by the language model from the statistics and from real excerpts of reviews. It has three columns: strengths, points to improve and three priority actions.

The Insights section holds five charts. Sentiment of written reviews splits them into positive, neutral and negative. Share of mentions by theme shows which topics dominate the conversation. Rating and complaints over time draws the average stars per year or per month as a line, and the share of one and two-star ratings as bars, so you can see when things got worse or better. The priority matrix places each theme by how often it is mentioned (left to right) and by net sentiment, the share of positive minus negative reviews (bottom to top). Themes in the bottom right corner, Fix first, are mentioned often and mostly negatively: they deserve attention first. Each dot carries a number, and the list under the chart gives the theme, its number of mentions and its net score. Average stars by theme shows the stars customers gave when they mentioned each theme.

What people talk about shows, for each theme, how many reviews mention it and how they split between positive, neutral and negative. Star ratings shows the distribution of stars and the measured reliability of the AI on these reviews, compared with a naive baseline.

Complaints without an answer lists the negative reviews that received no reply, newest first. The latest ones come with an AI reply draft. Long reviews are shortened, and Read the full review shows the whole text. Show the other complaints opens the rest of the list. When the file contains no owner reply at all, a note warns that either the business never answers or the export did not include replies, which you can check on Google Maps.

All reviews is a table of every review in the file, written or star-only, with its date, stars, text, AI sentiment, themes and whether the owner answered. It is sorted from newest to oldest and folds with one click.

5. Filter, draft replies and export

The filters above the dashboard (Period, Stars and Theme) recompute every figure, chart, list and table from the reviews already analyzed, without running the models again, so they respond instantly. For example, choose 2025, 1 to 2 stars and admissions to read only last year's negative reviews about

admissions. When a theme is selected, star-only ratings disappear from the table because they have no text and therefore no theme. Reset filters returns to the full view. The AI diagnosis always describes the whole dataset.

The Reply assistant, under the dashboard, lists the negative reviews of the current view, unanswered ones first. Pick one, read it, and click Draft a reply. The draft appears in an editable box with a copy button, and in the review's card on the dashboard. Clicking again writes a new version. Always read and adapt a draft before posting it: it is a suggestion, not a verified answer.

Model evaluation details how often the AI agrees with the stars customers gave: accuracy on the three sentiment classes compared with a naive baseline, exact star matches, matches within one star, mean error, precision and recall per class, and a confusion matrix.

Search reviews and export contains the full review table with a search box (type "parking" or "admission" to find every review that mentions it), a download of the analyzed reviews as CSV, and the Build the HTML report button described in section 7.

6. Analyze your own business

To collect the Google reviews of any business, create an account on https://outscraper.com and open the Google Maps Reviews service. Enter the business name and city, or paste its Google Maps link, set the number of reviews, the language (`fr` for a French business) and the sorting by newest, then start the task. Download the result as Excel or CSV. Drop this file directly in the application.

Before sharing review data or adding it to the GitHub repository, convert it with the collection script. The raw Outscraper export contains reviewers' names, profile links and photos, and the script keeps only the place, date, stars, text and owner reply:

```
python scripts/collect_reviews_outscraper.py --file Outscraper-export.xlsx
```

The anonymized file is saved in `data/`. In Colab, you can run this command in a new cell after uploading the export with the file panel on the left, adding a `!` before `python`.

The script can also collect reviews directly through the Outscraper API, with a key from your Outscraper profile. Never write the key in the code or commit it to GitHub:

```
pip install outscraper
export OUTSCRAPER_API_KEY="your_key"
python scripts/collect_reviews_outscraper.py --query "ESSEC Business School, Cergy" --limit
500
```

Other sources work as long as the file has a column of review text. Column names are recognized automatically: the text can be called `text`, `review_text`, `review`, `comment` or `avis`; the stars `review_rating`, `stars`, `rating` or `note` (values such as 4, 4/5 or "4 stars"); the date `date` or `review_datetime_utc`; the owner reply `owner_answer` or `reply`. Comma and semicolon separators both work. Stars, dates and replies are optional, but without them the evaluation, the trend chart or the unanswered-review detection cannot be computed, and the dashboard says so.

7. Publish a report with a permanent link

Each analysis can become a static web page with the same dashboard, plus the model evaluation, which anyone can open without Colab. In the application, open Search reviews and export, choose how many reply drafts to include and click Build the HTML report, then download the file. In the notebook, section 6

does the same and downloads a `site.zip` containing the report and the updated home page.

On GitHub, upload the report to the `docs/reports/` folder of the repository and the home page `index.html` to `docs/`. The first time only, activate GitHub Pages in `Settings > Pages` with the source Deploy from a branch, the branch `main` and the folder `/docs`. One or two minutes later the report is online at `https://monka31.github.io/reviewpulse/`, which lists every published report.

8. Run ReviewPulse on your own computer

This option suits people comfortable with a terminal. You need Python 3.10 or newer and about 8 GB of free disk space for the models. Without an NVIDIA GPU, the application works but text generation is slow.

```
git clone https://github.com/Monka31/reviewpulse.git
cd reviewpulse
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install torch                  # see https://pytorch.org for the GPU version
pip install -r requirements.txt
python app.py                      # then open http://127.0.0.1:7860
```

The command line gives the same analysis without the interface. It prints the main figures and writes the analyzed reviews to `outputs/` and an HTML report to `docs/reports/`:

```
python scripts/analyze.py data/reviews_ESSEC_Cergy.csv --name "ESSEC Business School" --type
"School / University" --replies 5
```

`python scripts/analyze.py --help` lists every option, including the summary language and custom themes.

9. Troubleshooting

If the notebook says there is no GPU, or generation is very slow, choose `Runtime > Change runtime type > T4 GPU`, then run everything again. If Colab cannot connect to a GPU, free GPUs are probably all in use: wait a little and retry, or run on a CPU and accept slower generation.

If the setup cell stops with "Code not found", the copy of the repository on GitHub is incomplete. Check that the `reviewpulse/` folder of the repository contains its ten Python files, then use `Runtime > Disconnect and delete runtime` and run everything again.

If the application says that no review text column was found, rename the column that holds the reviews to `text`. If accents look broken when you open a CSV in Excel, import it through `Data > From Text/CSV` and choose UTF-8. If the `gradio.live` link stops working, the Colab session has ended: run the last cell again. If the GPU runs out of memory, lower Max reviews in the advanced settings.

10. Generate the demo video automatically

Section 8 of the notebook builds the demo video with no manual step. Set `MAKE_DEMO_VIDEO = True` in that cell and run it, or run everything. It takes about fifteen minutes on a T4 GPU and downloads `ReviewPulse_demo.mp4` at the end.

The cell analyzes the ESSEC reviews, writes the narration from the real results (ratings, share of negative reviews, unanswered complaints, the theme to fix first, the first priority action, the measured accuracy), and has it read by an AI voice. A headless browser then opens the GitHub repository, the notebook and the application, uploads the file, starts the analysis, scrolls through the dashboard, applies a filter, drafts a

reply, opens the table and the model evaluation, and shows the published report. Each shot lasts exactly as long as its sentence, and the waiting time of the analysis is cut. ffmpeg assembles a 1080p video with subtitles, between 4:42 and 5:00.

`VOICE_ENGINE = "edge"` uses Microsoft neural voices through edge-tts and is the recommended setting. `"kokoro"` uses the open-source Kokoro voice model from Hugging Face. `VOICE` sets the voice, for example `en-US-AvaNeural`, `en-US-AndrewNeural` or `en-GB-SoniaNeural`. When Colab does not let a browser open the notebook without signing in, the video shows the content of the notebook instead.

From a terminal, the same video comes from `python scripts/make_demo_video.py`, after `pip install playwright edge-tts` and `playwright install --with-deps chromium`.

11. For developers

All settings (models, theme presets, thresholds) are in `reviewpulse/config.py`. Models can be replaced without touching the code through the environment variables `REVIEWPULSE_SENTIMENT_MODEL`, `REVIEWPULSE_ZERO_SHOT_MODEL` and `REVIEWPULSE_LLM_MODEL`. Every prompt sent to the language model is in `reviewpulse/llm.py`. The analysis can also be called from Python with `reviewpulse.run_pipeline(path, business_type="Restaurant")`.

The tests run in a few seconds without downloading any model, using a keyword-based mock backend that is enabled only for testing and must never be used for a real analysis:

```
pip install pytest
pytest -q
```

Technical report

This report explains how ReviewPulse works, why each tool was chosen, how its outputs are evaluated and what its limits are. The user guide covers how to use it.

1. Objective and scope

ReviewPulse takes the Google reviews of one business and produces a dashboard that an owner can act on: what customers like, what they complain about, how this changes over time, which complaints were left unanswered and how to reply to them. The target users are independent businesses with a few dozen to a few thousand reviews, such as restaurants, shops, hotels or schools, that cannot afford a customer-experience platform.

Three constraints shaped the design. The application had to be free to run, so it relies on open-source models and a free Colab GPU rather than a paid API. It had to be reproducible by a grader in one click, so everything starts from a single notebook and the data is stored in the repository. And it had to be built in a few days by a small team, so it reuses pre-trained models instead of training new ones.

2. Architecture

The code is a small Python package, `reviewpulse/`, used by three entry points that share the same pipeline: the Colab notebook, the Gradio web application (`app.py`) and a command-line script (`scripts/analyze.py`). One function, `run_pipeline`, loads a file, runs the analysis, asks the language model for the diagnosis and the reply drafts, and returns a result object that the dashboard, the evaluation and the report export all read.

The pipeline has five stages. Loading and cleaning turns any review file into one standard table. Sentiment analysis predicts a star rating for each written review. Theme detection assigns each review to the business's themes. Aggregation computes statistics per theme and over time. Generation writes the diagnosis and the replies. A sixth step, evaluation, compares the predicted sentiment with the stars customers gave.

The models are loaded once per session, on first use, and kept in memory, so a second analysis in the same session skips the download and loading time.

3. Data loading and cleaning (`data.py`)

Review files come from different sources with different column names, so the loader looks for known names in order of priority. The text can be called `text`, `review_text`, `review`, `comment` or `avis`, the stars `review_rating`, `stars`, `rating` or `note`, and so on. Only the text column is required.

The order matters for Outscraper exports. Their Excel and CSV files repeat place-level columns on every row: `rating` there is the average of the business (4.4 for ESSEC), and `reviews` is its total number of reviews. Matching `review_rating` before `rating`, and never treating `reviews` as text, avoids silently using the business average as every review's score. An automated test checks this case.

Columns that identify reviewers (name, profile link, identifier, photo) are dropped as soon as the file is read. HTML left by the export, such as `<br>` tags and `&` entities, is converted to plain text. Stars written as "4", "4/5" or "4 stars" are all read as 4. The separator (comma, semicolon or tab) is detected from the header line, so French Excel exports work.

Reviews without text are common on Google: 174 of the 250 ESSEC ratings have no text. They cannot be analyzed by the language models, but they matter for the public average and for the trend over time. The loader therefore removes them from the text analysis but keeps their date, stars and reply status aside, so that the average rating, the trend chart and the table of all reviews still include them. Empty texts, texts shorter than three characters and exact duplicates are removed, and reviews are sorted from newest to oldest.

4. Sentiment analysis

Each written review goes through `nlptown/bert-base-multilingual-uncased-sentiment`, a BERT model fine-tuned on product reviews in English, Dutch, German, French, Spanish and Italian. It predicts a rating from one to five stars with a confidence score. We map one and two stars to negative, three to neutral and four and five to positive.

We chose this model because it handles French and English, the two languages of our datasets, and because its output is in stars, the same unit as the customer ratings. That makes the evaluation in section 8 direct. Reviews are truncated to 512 tokens and processed in batches of 32.

5. Theme detection with zero-shot classification

Themes are detected with `MoritzLaurer/mDeBERTa-v3-base-mnli-xnli`, a multilingual model trained for natural language inference. For each review and each theme, the model reads the review as a premise and the sentence "This review is about {theme}." as a hypothesis, and returns the probability that the review entails it. A review gets every theme whose probability reaches 0.5, so one review can mention several themes, and its main theme is the one with the highest score.

Zero-shot classification lets a business choose its own themes without labeling any example, which is why the application offers editable presets per business type. We preferred this multilingual model to the more common `facebook/bart-large-mnli`, which is trained on English only, because many reviews are in French. The hypothesis stays in English, and the multilingual model matches it against reviews in either language.

The cost of zero-shot classification grows with the number of reviews times the number of themes, since every pair is a separate inference. Presets therefore hold six to eight themes, users are limited to ten, and reviews longer than 1,200 characters are truncated for this step only.

6. Aggregation and insights

For each theme, the application counts the reviews that mention it and splits them into positive, neutral and negative. The net score is the share of positive reviews minus the share of negative ones, from -100 to +100. A theme needs at least three mentions to be named a strength or a pain point, so that one angry review does not define the diagnosis.

The trend chart groups all ratings, star-only ones included, by year when the data spans more than two years and by month otherwise. Periods with fewer than three ratings are hidden and named under the chart, because an average of one or two ratings is not meaningful.

The priority matrix places each theme by its number of mentions on the horizontal axis and its net score on the vertical axis. The four quadrants read as Fix first (frequent and negative), Strengths to promote (frequent and positive), Niche strengths and Watch. To keep the chart readable with any data, dots carry a number and the theme names are listed under the chart. Dots that would overlap are moved slightly apart.

When a file has an owner-reply column, a negative review is counted as unanswered when its reply is empty. A review counts as negative when the model reads it as negative or when its author gave one or two stars, so that a complaint the model misreads still appears in the list.

7. Text generation (`llm.py`)

The diagnosis and the reply drafts are written by `Qwen/Qwen2.5-3B-Instruct`, a small open-source instruction model. With 3 billion parameters in half precision, it fits on the free T4 GPU of Colab, and it is the largest size of the Qwen2.5 family that leaves room for the two other models. We chose it over the 1.5-billion version because the replies are shown to customers, so the quality of the French matters more than a few seconds of generation. Without a GPU, the application switches to `Qwen2.5-0.5B-Instruct` so that it remains usable. Generation is deterministic (greedy decoding), so the same data and settings give the same text, which helps reproducibility.

The model does not read the reviews in bulk. For the diagnosis, it receives the computed statistics (sentiment split, average rating, reply rate, mentions and net score per theme) and a sample of real excerpts: the eight negative and five positive reviews the sentiment model is most confident about. The prompt asks for three Markdown sections (strengths, areas for improvement, three priority actions) in under 220 words, with each point tied to a theme, and tells the model never to state facts the data does not support. Giving the model numbers it would otherwise have to guess, and asking it to cite themes, limits hallucinations and makes the output easy to check against the charts.

For each reply, the model receives the review and its stars, with explicit rules: write in the language of the review, between 60 and 110 words, thank the customer and acknowledge the specific issue, never blame the customer, never promise refunds or discounts, never invent facts, invite direct contact, and sign as the team. Replies are drafted for the five most recent unanswered complaints during the analysis and on demand for the others, which keeps the analysis short. The full prompts are reproduced in `reviewpulse/llm.py` and summarized in the prompts log.

8. Evaluation method (`evaluation.py`)

Every Google review comes with the stars its author gave. These stars are labels we did not have to produce, so we use them to measure the sentiment model on each business's own reviews.

The application reports accuracy on the three sentiment classes, compared with a baseline that always predicts the most frequent class. On a business with mostly positive reviews, a useless model that always says "positive" already reaches a high accuracy, so the model is only useful if it beats that baseline. It also reports the share of exact star matches, the share within one star, the mean absolute error in stars, precision and recall for each class, and a confusion matrix of actual against predicted stars.

Two caveats apply. Customer stars are a noisy label: some people write a critical text and still give four stars, or the reverse. And three-star reviews are usually the hardest, because their texts mix praise and criticism. The evaluation results on the demo datasets are reported in section 13.

9. Interface and dashboard

The application uses Gradio for the controls (file upload, settings, filters, reply assistant) and renders the dashboard itself as HTML and CSS, with charts drawn as inline SVG by Python. This gives a layout built around what an owner needs to see, and the same code produces the dashboard in the application, in the notebook and in the static report, with no JavaScript. The visual identity uses a light blue close to ESSEC's, Inter for text and Plus Jakarta Sans for titles.

Filters re-slice the reviews already analyzed and redraw the dashboard, without calling the models again, which is why they respond instantly. Interactive Plotly versions of the main charts remain in the notebook for exploration.

10. Reproducibility

The notebook is the reference way to run the project. It clones the repository into a folder named `reviewpulse_app` (a name different from the Python package, to avoid an import clash we met during development), pulls the latest version, installs pinned dependency ranges from `requirements.txt`, checks that the package loads correctly and runs the analysis on the dataset stored in `data/`. Models are downloaded by name from Hugging Face. Text generation is deterministic.

The test suite (`tests/test_pipeline.py`) runs in a few seconds without any model, using a keyword-based mock backend that exists only for testing. It covers file loading for several formats (Outscraper exports, semicolon CSV, text-only files), the anonymization, the place-level rating trap, the full pipeline, the filters, the table of all reviews, the report export and the construction of the application. The real model code path was also run end to end with tiny randomly initialized models before the first Colab run.

11. Automated demo video (`demo_video.py`)

The demo video is produced by the project itself. The narration is a template filled with the figures of the analysis, so it never quotes a number the application did not compute; when the model does worse than the naive baseline, the narration says so. The voice comes from edge-tts or from the Kokoro model. Instead of recording the screen in real time, a headless Chromium driven by Playwright takes one screenshot per animation step (fifteen per second for scrolling and cursor moves) and holds the last frame of each shot for as long as its sentence lasts. Time on screen is therefore independent of how long the models take, which removes the waiting time and keeps the video between 4:42 and 5:00. If the narration runs long, the voice speeds up slightly; if it runs short, the pauses between shots grow. ffmpeg then joins the frames, the narration track and subtitles generated from the same text.

12. Alternatives considered

We compared several ideas before choosing review analysis, including a sales-call objection coach based on speech transcription and a support-ticket triage tool. Review analysis won because it needs no training, uses public data, has an obvious business value and makes a visual demo.

For the data, the Yelp Open Dataset offered millions of reviews but almost no French businesses. The official Google Places API returns only five reviews per place. A Chrome extension reading the review page in the browser would have made a striking demo, but it was longer to build and much harder for a grader to reproduce. We chose Outscraper exports, which give every review of a place with its stars, date and owner reply.

For hosting, a site on Vercel with a Supabase database would have given a permanent application, but it added two services, credentials and points of failure a few days before the deadline, and made reproduction harder. Hugging Face Spaces offers a permanent link but no free GPU, which makes text generation very slow. We kept Colab for the application and used GitHub Pages for permanent static reports, which needs no server at all.

13. Results on the demo datasets

On the ESSEC Cergy campus (250 ratings collected on October 25, 2026, 76 of them with text), the data alone shows a public average of 4.28 out of 5, an average of 3.66 among written reviews, 22 written reviews at one star, no owner reply in the export, and a dip in 2023 (3.9 out of 5, 28% of ratings at one or two stars) followed by a recovery to 4.5 in 2026.

[Add after the final Colab run: model accuracy against customer stars and the majority-class baseline, main themes and quadrant of the priority matrix, number of unanswered complaints, analysis time on a T4 GPU, and the same figures for the restaurant.]

14. Ethics and privacy

Reviewer names, identifiers, profile links and photos are removed before any analysis and never stored in the repository. Only the text, stars, date and owner reply are kept, and only for analysis. Reply drafts are labeled as AI drafts to be checked by a person, and the application never posts anything. The ESSEC case is presented constructively, as an example of what a school could improve. Large-scale collection of Google Maps data may conflict with Google's terms of service, so the project is an academic prototype and not a scraping service.

15. Limitations and next steps

The sentiment model misreads irony and mixed reviews, and its accuracy depends on the business, which is why it is measured each time. Zero-shot themes depend on wording, and a theme such as "experience" catches almost everything. The small language model can produce generic replies. Colab sessions are temporary and free GPUs are not guaranteed.

The next steps would be aspect-based sentiment (the sentiment of each theme within a review, instead of one sentiment per review), a larger or fine-tuned language model for replies, automatic periodic collection with alerts on new negative reviews, a comparison view between several branches or competitors, and a permanent hosted version with a GPU.

Data

Datasets in the repository

`data/reviews_ESSEC_Cergy.csv` holds 250 Google Maps reviews of the ESSEC Business School Cergy-Pontoise campus, collected with Outscraper on October 25, 2026. They run from November 2020 to October 2026. 76 reviews have a text and 174 are star-only ratings. The Google Maps listing showed 382 reviews in total at the time of collection, so the file covers about two thirds of them. The export contained no reply from the school to any review.

`data/reviews_<restaurant>.csv` [to add: the restaurant used in the second demo, collected the same way.]

`data/demo_restaurant_synthetic.csv` holds 30 reviews of a fictional restaurant, "Demo Bistro", written by the team to test the application. They are synthetic, labeled as such in the file, and never presented as real data.

Format

Every dataset uses the same five columns. `place` is the name of the business. `date` is the publication date of the review (UTC). `rating` is the number of stars given by the author, from 1 to 5. `text` is the review, empty for star-only ratings. `owner_answer` is the public reply of the business, empty when there is none.

The application also reads other formats, such as raw Outscraper exports, Kaggle datasets or a spreadsheet typed by hand. The user guide lists the column names it recognizes.

Collection

Reviews were collected with the Outscraper Google Maps Reviews service (https://outscraper.com), which returns the reviews of a place with their text, stars, date and owner reply. The export was converted with `scripts/collect_reviews_outscraper.py`, which keeps the five columns above and discards everything else. The same script can collect reviews directly through the Outscraper API with a personal key, read from an environment variable and never stored in the code.

Before choosing Outscraper, we looked at the Yelp Open Dataset on Kaggle, which has almost no French businesses, and at the official Google Places API, which returns only five reviews per place.

Privacy

The raw Outscraper export contains the name, profile link, identifier and photo of each reviewer. None of these are kept: the collection script drops them, and the application also drops them when a raw export is loaded directly. The repository contains no personal data about reviewers. The data is used for analysis only.

Google reviews are public, but large-scale automated collection may conflict with Google's terms of service. This project is an academic prototype built on a small number of places, not a data collection service.

AI tools and prompts log

The assignment asks us to show where and how we used AI tools, with the prompts. This log covers the AI assistants used to design and build ReviewPulse, then the prompts the application itself sends to its language model.

1. AI assistants used

We used Claude (Anthropic, on claude.ai) throughout the project: to choose the idea and the data source, to compare architectures, to write and debug the code, to design the interface and to draft the documentation and the video script. [Add any other tool the team used, for example ChatGPT or GitHub Copilot, with what it was used for.]

The prompts were written in French. Each one is quoted in French, followed by an English translation and the outcome. Every output was read, tested and corrected by the team before being kept. When code failed in Colab, we pasted the error message back and asked for a fix, so several prompts below are error reports.

2. Prompts used to build the project

Understanding the assignment and organizing the team

Prompt 1. We uploaded the assignment PDF with "J'ai ce projet : [instructions de l'enseignant]" (I have this project: [teacher's instructions]). The assistant summarized the grading scheme, pointed out that reproducibility, duration compliance and documentation were points we could secure early, and suggested three first ideas: a customer-success copilot, a sales objection coach and a review analyzer.

Prompts 2 and 3. "Je vais créer un groupe avec tous les membres, fais-moi le message de bienvenue, fais tout en anglais en demandant si tout le monde est francophone. Demande comment ils veulent se répartir la tâche" (I am creating a group chat with all members, write the welcome message in English, ask whether everyone speaks French and how they want to split the work), then "S'ils ont déjà des idées etc" (and whether they already have ideas). The assistant drafted the welcome message with a proposed split into four roles.

Choosing the idea

Prompt 4. "Quel projet pourrait être intéressant à faire, facile, rapide, utile, et qui utiliserait les bibliothèques présentées ?" (Which project would be interesting, easy, fast, useful, and use the presented libraries?) The assistant recommended a review analyzer built from pre-trained Hugging Face models, with a demo outline.

Prompt 5. "Peux-tu me faire plus de propositions de projet stp" (Can you give me more project ideas?) It proposed eleven ideas across marketing, sales, document processing and computer vision.

Prompt 6. "Donne une idée simple qui réponde aux critères, il faut que je puisse le faire rapidement" (Give one simple idea that meets the criteria, I must be able to build it quickly). It confirmed the review analyzer and gave a one-day build plan.

Finding the data

Prompt 7. "Puis-je de mon côté récupérer les infos de n'importe quel magasin ou commerce pour ma démo ?" (Can I get the data of any store or business for my demo?) The assistant compared copying reviews by hand, the Google Places API (five reviews per place), open datasets and scraping, with the legal and privacy caveats.

Prompt 8. "Montre-moi comment avoir un fichier CSV avec tous les avis d'une entreprise, commerce, restaurant peu importe" (Show me how to get a CSV with all the reviews of any business). It wrote a script extracting one business from the Yelp Open Dataset. We did not use it, because Yelp has almost no French businesses.

Prompt 9. "Je veux le faire avec des commerces français, par exemple, super exemple, je veux le faire avec tous les avis qu'il y a sur l'ESSEC" (I want French businesses, for example all the reviews about ESSEC). It listed where ESSEC reviews can be found and how to collect them.

Prompt 10. "Est-il possible de faire une extension Chrome qui, en fonction de la page d'avis où l'on est, analyse tout seul ?" (Could a Chrome extension analyze the review page you are on?) It explained that it was feasible but longer to build and harder for graders to reproduce. We dropped the idea: "ok on oublie, faisons quelque chose de simple où l'on fournit un fichier CSV" (let's forget it and keep it simple, with a CSV file).

Prompts 11 and 12. "Peut-on le faire avec Apify ?" (Can we do it with Apify?), with the documentation of an Apify actor, then "ok faisons avec Outscraper : [exemple de sortie JSON]" (let's use Outscraper: [sample JSON output]). The assistant wrote the collection script that keeps only the text, stars, date and owner reply and drops every reviewer identity. It later became `scripts/collect_reviews_outscraper.py`.

Explaining the project to the group

Prompts 13 to 15. "Peux-tu me faire le texte d'explication pour mon groupe pour leur expliquer le projet, l'intérêt etc, que pendant la démo on peut faire un test avec l'ESSEC, un test avec un restaurant afin de montrer les axes d'amélioration, les messages non répondus" (write a message explaining the project and its value to the group, with a demo testing ESSEC and a restaurant to show improvement areas and unanswered reviews), then a French version, then "Fais un message plus court sans emoji, qui ne fait pas IA" (make it shorter, without emojis, so it does not sound AI-written).

Architecture decisions

Prompt 16. "Partons sur ce projet, que devons-nous faire ? La structure ? Quoi utiliser ? Tout est gratuit ? Dépôt GitHub ?" (What do we need to do, what structure, which tools, is everything free, GitHub repository?) The assistant proposed the repository structure, the free stack and a planning.

Prompts 17 to 19. "On fait quelque chose en local ou en ligne ?" (local or online?), "Comment se compose l'appli ? Comment est-elle hébergée, comment elle tourne ?" (what are the parts of the app, how is it hosted and run?) and "C'est un peu nul que ce soit périssable, ne peut-on pas faire quelque chose sur Supabase, Vercel ou autre pour avoir un lien permanent ?" (the temporary link is a pity, could we use Supabase or Vercel for a permanent link?). The assistant explained the Colab and Gradio setup, then compared Vercel with Supabase and Hugging Face Spaces.

Prompts 20 and 21. "Si on part sur Google Colab, combien de temps prendrait une analyse ?" (how long would an analysis take on Colab?) and "Donc concrètement, avec du recul et de la critique, que devons-nous faire ?" (stepping back critically, what should we do?). The assistant advised against the extra

infrastructure, recommended Colab with Gradio for the application and GitHub Pages for permanent static reports, an evaluation against customer stars, and a 3-billion-parameter language model for better French. We followed this advice.

Building the application

Prompt 22. "Fais l'appli complète, fais tout ce que tu peux faire, pense à tout, mets-toi à la place d'un développeur senior qui doit rendre la chose simple et facilement reproductible. Ensuite dis-moi ce qu'il me manque à faire." (Build the complete app, think of everything, act as a senior developer who must keep it simple and easily reproducible, then tell me what is left for me to do.) The assistant wrote the package, the Gradio application, the notebook, the report export, the tests and the first documentation, checked them with automated tests and an end-to-end run using tiny local models, and listed the remaining tasks.

Prompts 23 to 25. "Fais-moi la description du repo GitHub et tous les paramètres à activer ou non" (write the repository description and tell me which settings to enable), "Comment le déployer et utiliser l'outil ? Je ne vois pas Website" (how do I deploy and use it? I cannot find the Website field) and "Quoi mettre ici" (what goes here?) with a screenshot of the repository settings.

Prompt 26. "Voici un exemple de template avis (c'est celui de l'ESSEC), l'app est optimisée pour ?" (here is a review export, the ESSEC one: is the app optimized for it?), with the Outscraper Excel file. Testing it revealed three problems: the app read the place-level average (4.4) instead of each review's stars, kept HTML tags in the texts, and ignored star-only ratings. The assistant fixed all three, added a regression test and adjusted the school themes to what the reviews discuss (admissions, staff and welcome).

Prompts 27 to 29. Questions on how to use the app ("Comment fonctionne l'appli ? Je dois déposer les avis sur GitHub ? Je ne peux pas poser le Excel sur l'appli directement ?", how does it work, do I have to put the reviews on GitHub, can't I drop the Excel file directly?) and error reports from Colab: "Could not find ReviewPulse.ipynb", then "FileNotFoundError: data/reviews_ESSEC_Cergy.csv". The causes were an upload that had put the files in a subfolder, then a stale copy of the repository in Colab. The notebook now pulls the latest version on every run.

Redesigning the interface

Prompt 30. "Je n'aime pas du tout l'interface de l'appli, peux-tu m'aider à changer le template ? Cela marche en HTML ? Comment ? Ne fais pas de modif avant ma validation" (I do not like the interface, can you change it, does it work with HTML, do not change anything before I approve). The assistant proposed three options and we chose the second: Gradio controls with an HTML dashboard ("ok partons sur l'option B si elle est facile de mise en place").

Prompts 31 to 33. The assistant drew a mockup of the dashboard for validation. We asked for "le bleu clair ESSEC" (ESSEC light blue), then "Peut-on ajouter des graphes ? J'aimerais pouvoir voir des camemberts etc ou d'autres graphes en fonction d'infos ou données particulières. Interface en anglais" (can we add charts, such as pie charts, depending on specific information? Interface in English). It added the five charts of the Insights section and the period, stars and theme filters. We approved with "go".

Prompts 34 to 36. Error reports after the redesign: "AttributeError: module 'reviewpulse' has no attribute 'run_pipeline'", twice, then an assertion error. The first cause was a name clash between the folder Colab cloned and the Python package, fixed by cloning into `reviewpulse_app`. The second was an incomplete upload to GitHub.

Prompt 37. "Penses-tu cela possible de faire en sorte qu'aucun texte ne se superpose ? Est-il également possible d'ajouter un tableau avec tous les avis ? Répondus ou pas, avec ou sans texte, tout dans un tableau" (can you make sure no text overlaps, with a screenshot of the priority matrix, and add a table

with every review, answered or not, with or without text?). The matrix now uses numbered dots with a list under the chart, and the dashboard ends with a table of every review.

Documentation

Prompt 38. "Peux-tu stp me faire toute la doc explicative, tout tout tout, répondant au cahier des charges" (write all the documentation, everything, matching the assignment), with a request to make the text sound human rather than AI-generated. The assistant wrote this log, the README, the user guide, the technical report, the data notes and the updated video script.

Automated demo video

Prompts 39 to 43. "Penses-tu qu'il serait possible de faire une démo automatisée avec une voix off qui explique tout ?" (could the demo be automated, with a voice-over explaining everything?), "Peut-on le faire uniquement avec le fichier ESSEC ? Cela suffit-il ?" (can we do it with the ESSEC file only, is that enough?), "Comment automatiser la démo ?" (how would we automate it?), "Je veux tout automatiser à 100 %, est-ce possible ?" (I want it 100% automated, is that possible?) and "Je veux le faire uniquement avec les données de l'ESSEC, automatise tout, je veux que la démo puisse montrer le GitHub, le Google Colab, lance l'analyse, lance l'outil, défile tout, explique tout" (only the ESSEC data, automate everything, show GitHub and Colab, run the analysis, open the tool, scroll through everything and explain it all). The assistant compared a fully automated video with a recording narrated by the team, pointed out that a synthetic voice could cost points on delivery, and built `reviewpulse/demo_video.py` and section 8 of the notebook. The narration is written from the analysis results, so the video quotes only figures the application computed.

[Add the prompts used after this point.]

3. Prompts used inside the application

ReviewPulse sends two kinds of prompts to `Qwen/Qwen2.5-3B-Instruct`. Their exact text is in `reviewpulse/llm.py`.

The diagnosis prompt uses this system message: "You are a customer-experience consultant. You analyze online reviews for a business owner and give short, concrete, evidence-based advice. Never invent facts that are not supported by the data or the review excerpts you are given." The user message gives the business name and type, the number of reviews, the sentiment split, the average rating and reply rate, the mentions and net score of each theme, eight negative and five positive review excerpts, and asks for three Markdown sections (strengths, areas for improvement, three priority actions) in under 220 words and in the chosen language.

The reply prompt uses this system message: "You write public replies to customer reviews on behalf of a business. Replies are warm, professional, specific to what the customer wrote, and never defensive." The user message gives the business, the review and its stars, with these rules: write in the same language as the review, 60 to 110 words, thank the customer and acknowledge the specific issue, say what will be done, never blame the customer, never promise refunds or discounts, never invent facts, invite direct contact and sign as the business team.

We wrote the prompts this way so the model works from computed figures and real excerpts instead of guessing, follows a fixed output format that the dashboard can display, and stays within limits a business could accept. Generation is deterministic, so the same data gives the same text.